

Ausarbeitung Projekt

Data Science Virtueller Tutor Elektronik-Labor

Joannis Kopp

Philip Lesche

Jannis Fink

16. August 2026

Contents

1	Methodik	3
1.1	Untersuchungsdesign	3
1.2	Untersuchungsgegenstand und Datengrundlage	3
1.3	Evaluationsarchitektur	4
1.4	Verwendete Methoden und Werkzeuge	4
1.5	Bewertungskriterien	5
1.5.1	Sokratische Qualität (RQ1)	5
1.5.2	RetrievalQualität (RQ2)	5
1.5.3	Aggregation und GateLogik	5
1.6	Durchführung	6
1.7	Gütekriterien und methodische Einschränkungen	6

1 Methodik

Gegenstand der vorliegenden Untersuchung ist die Evaluation eines sokratischen KITutors, der auf einer RetrievalAugmentedGenerationPipeline (RAG) aufsetzt. Ziel ist es, das Zusammenspiel von Prompting und RetrievalKonfiguration systematisch zu bewerten. Die Untersuchung folgt einem explorativen, experimentellen Evaluationsdesign, das zwei Forschungsfragen getrennt adressiert: den Einfluss verschiedener sokratischer Prompts (RQ1) sowie den Einfluss verschiedener RAGKonfigurationen (RQ2). Beide Fragen werden über dieselbe technische Evaluationsarchitektur beantwortet, jedoch in zwei voneinander unabhängigen Auswertungslinien, sodass jeweils nur ein Faktor variiert wird.

1.1 Untersuchungsdesign

Die Evaluation ist als automatisierte, mehrstufige Dialogbewertung angelegt. Für jedes Testszenario wird ein MultiTurnGespräch zwischen einer simulierten studierenden Person und dem RAGgestützten Tutor erzeugt und anschließend anhand eines definierten MetrikSets bewertet. Die Wahl eines simulationsbasierten MultiTurnDesigns ergibt sich unmittelbar aus dem Untersuchungsgegenstand: Da sokratisches Tutoring ein dialogisches, über mehrere Gesprächsrunden verlaufendes Vorgehen ist, lässt es sich nicht anhand einzelner FrageAntwortPaare, sondern nur im Verlauf eines Gesprächs angemessen beurteilen **QUELLE _SOKRATIK**.

Zur klaren Trennung der Forschungsfragen wurden zwei separate Evaluationslinien umgesetzt. In der ersten Linie (RQ1) wird bei konstant gehaltener RAGKonfiguration der SystemPrompt variiert; bewertet wird die *sokratische Qualität* des Gesprächsverlaufs. In der zweiten Linie (RQ2) wird bei konstant gehaltenem Prompt die RAGKonfiguration variiert; bewertet wird die *RetrievalQualität*. Durch die Beschränkung auf jeweils einen variierten Faktor je Linie können die beobachteten Effekte den Prompts bzw. den Konfigurationen zugeordnet werden.

1.2 Untersuchungsgegenstand und Datengrundlage

Das zu evaluierende System (System under Test) ist eine OpenWebUIkompatible RAG-Pipeline, die einen QdrantVektorspeicher, ein CrossEncoderReranking sowie Moodle-Metadaten anbindet. Als generierendes Sprachmodell nutzt die Pipeline intern das Modell `gemma426B`. Der fachliche Wissenskorpus liegt in zwei QdrantCollections vor, die Vorlesungs- bzw. Labormaterial des Fachgebiets [PLATZHALTER: Fachgebiet/Themen konkretisieren] enthalten.

Die Datengrundlage der Evaluation bilden nicht vorab gesammelte Dialoge, sondern *synthetisch erzeugte* Gespräche. Grundlage jedes Gesprächs ist ein Testszenario, das sich aus der Kombination von Themengebiet, Kompetenzniveau der studierenden Person (An-

fänger bzw. Fortgeschritten) und einem charakteristischen StudierendenVerhalten (etwa kooperativ, die Lösung einfordernd oder inhaltlich selbstsicherfehlerhaft) zusammensetzt. Insgesamt ergeben sich daraus [PLATZHALTER: Anzahl] Szenarien je Auswertungslinie. Jedes Szenario wird als *Golden* modelliert, das die Ausgangssituation, eine Beschreibung der zu simulierenden Persona sowie eine feste Initialfrage enthält. Die Initialfrage wird bewusst wortgleich vorgegeben und nicht generiert, um über alle Prompts und Konfigurationen hinweg denselben Gesprächseinstieg und damit die Vergleichbarkeit der Läufe sicherzustellen; die daran anschließenden Folgefragen werden hingegen dynamisch simuliert.

1.3 Evaluationsarchitektur

Da der Untersuchungsgegenstand ein Dialog ist, sind neben dem zu bewertenden System zwei weitere Rollen erforderlich. Die Architektur umfasst somit drei klar getrennte Modelle. Der *Tutor* ist die RAGPipeline (`gemma426B`) und beantwortet jeden Gesprächszug. Der *StudentenSimulator* übernimmt die Rolle der studierenden Person und erzeugt auf Basis von Szenario, Persona und der jeweils vorangegangenen TutorAntwort die Folgefragen; ohne diese Rolle entstünde kein Dialog, sondern lediglich ein einzelnes Frage-AntwortPaar. Als Simulator wird das Modell `gpt4omini` eingesetzt, das auf dem verwendeten Endpunkt einen Alias für `gemma426B` darstellt und somit auf demselben Basismodell wie die RAGPipeline beruht. Der *Judge* bewertet den fertigen Dialog; hierfür wird mit `gpt4.1` bewusst ein vom System unter Test verschiedenes Modell gewählt, um einen SelbstbewertungsBias zu vermeiden, bei dem ein Modell anteilig seine eigenen Ausgaben bewertet **QUELLE_LLM_JUDGE**. Simulator und Judge laufen auf getrennten Endpunkten, was zugleich die kostengünstige lokale Simulation von der kostenpflichtigen JudgeBewertung entkoppelt.

1.4 Verwendete Methoden und Werkzeuge

Die technische Umsetzung erfolgt in Python unter Verwendung des EvaluationsFrameworks DeepEval **QUELLE_DEEPEVAL**. Die MultiTurnGespräche werden mit dem `ConversationSimulator` des Frameworks erzeugt: Ausgehend von der festen Initialfrage generiert der Simulator über eine festgelegte Anzahl von Runden fortlaufend StudierendenNachrichten, die über eine CallbackSchnittstelle an die RAGPipeline gesendet werden; deren Antwort bildet jeweils den nächsten AssistantTurn. Die Zuordnung von Antworten zu Metriken erfolgt anschließend über `ConversationalTestCase`Objekte des Frameworks.

Für die kontextbezogenen Metriken wird der tatsächlich abgerufene RetrievalKontext benötigt. Dieser wird über den VerboseModus der RAGPipeline gewonnen, der bei deaktiviertem Streaming eine strukturierte Antwort liefert, aus der die finalen RetrievalChunks samt Text extrahiert und den jeweiligen Antworten zugeordnet werden. Zwei Aufbere-

itungsschritte werden dabei transparent gemacht: Zum einen wird der Kontext für die Bewertung auf eine handhabbare Länge begrenzt, um ein Überschreiten des Kontextfensters des Judge zu vermeiden; zum anderen wird die mathematische Notation (LaTeX) für die Bewertung normalisiert, da sie andernfalls die strukturierte Ausgabe des Judge stört. Beide Eingriffe betreffen ausschließlich die dem Judge übergebene Kopie; die gespeicherten Gespräche bleiben unverändert.

1.5 Bewertungskriterien

Die Bewertung ist entlang der beiden Forschungsfragen unterschiedlich ausgestaltet.

1.5.1 Sokratische Qualität (RQ1)

Sokratisches Tutoring ist ein didaktikspezifisches Konstrukt, für das im verwendeten Framework keine vorgefertigten Metriken existieren. Es wird daher über eigene, mittels G-Eval **QUELLE_GEVAL** operationalisierte Kriterien bewertet, die jeweils über explizite Bewertungsschritte und eine dreistufige Bewertungsskala (Rubric) verankert sind. Verwendet werden die Kriterien *Lösung nicht verraten*, *Sokratische Rückfragen*, *Schrittweise Lernprogression*, *NiveauAnpassung*, *Fachliche Korrektheit*, *Verständnisförderung/Transfer* sowie *Respektvolle Kommunikation*. Ergänzend wird die im Framework enthaltene Metrik zur Rollentreue (*RoleAdherence*) als Gegenprüfung herangezogen, da sie die Treue zur im Testfall hinterlegten sokratischen Rollendefinition misst und damit als einzige native Metrik das Konstrukt indirekt berührt.

Bewusst nicht verwendet werden Metriken, die dem sokratischen Ziel widersprechen oder es verfehlen. Dies betrifft insbesondere eine Metrik zur direkten Zielerreichung, die die unmittelbare Beantwortung der Fachfrage bewertet und damit dem Prinzip, die Lösung gerade nicht vorwegzunehmen, entgegensteht, sowie eine generische TurnRelevanzMetrik, die legitimes sokratisches Umlenken als weniger relevant einstufen kann.

1.5.2 RetrievalQualität (RQ2)

Für den Vergleich der RAGKonfigurationen werden die nativen Metriken *TurnContextual-Relevancy* (Passung des abgerufenen Kontexts zur Frage) und *TurnFaithfulness* (Deckung der Antwort durch den abgerufenen Kontext) verwendet. Beide beziehen den zuvor extrahierten RetrievalKontext ein und eignen sich damit zur Unterscheidung von Retrieval-Konfigurationen.

1.5.3 Aggregation und GateLogik

Zur Beantwortung von RQ1 wird aus den G-EvalKriterien je Gespräch ein gewichteter Gesamtscore gebildet [PLATZHALTER: konkrete Gewichte angeben und begründen]. Da

eine Lösungspreisgabe für einen sokratischen Tutor kein untergeordneter Mangel, sondern eine Verletzung der Kernanforderung darstellt, werden die Kriterien *Lösung nicht verraten* und *Fachliche Korrektheit* zusätzlich als primäre, nicht durch andere Kriterien kompensierbare Anforderungen behandelt. Statt eines einfachen Mittelwerts, der eine solche Verletzung durch gute Werte in anderen Kriterien kaschieren würde, wird daher die Einhaltungsrates dieser Kernanforderungen als eigenständige Kennzahl berichtet. Für RQ2 werden die Konfigurationen anhand der Mittelwerte der RetrievalMetriken sowie der RetrievalAbdeckung, also des Anteils der Antworten mit tatsächlich abgerufenem Kontext, verglichen.

1.6 Durchführung

Die Durchführung erfolgt zweiphasig und je PromptVersion bzw. Konfiguration getrennt: Zunächst werden alle Gespräche simuliert, anschließend werden sie bewertet. Diese Trennung erlaubt es, die netz und laufzeitintensive Simulation gegen den RAGEndpoint einmalig durchzuführen und die JudgeBewertung unabhängig davon zu wiederholen. Zur Sicherstellung der Reproduzierbarkeit werden die simulierten Gespräche je Konversation atomar zwischengespeichert; über eine deterministische KonversationsID wird ein unterbrochener Lauf beim Neustart fortgesetzt, ohne bereits erzeugte Gespräche neu zu berechnen. Die Bewertungsergebnisse werden fortlaufend je Metrik protokolliert und ebenfalls fortsetzbar geschrieben.

Für RQ1 werden [PLATZHALTER: Anzahl] PromptVarianten bei einer festen RAG-Konfiguration ausgewertet. Für RQ2 wird ein fester sokratischer Prompt mit einer Matrix aus neun RAGKonfigurationen kombiniert, die sich in den einbezogenen Collections, im RetrievalModus (dense, sparse, hybrid) und im Einsatz des CrossEncoderRerankings unterscheiden und zusätzlich eine reine LLMKonfiguration ohne Retrieval als Referenz umfassen. Zur Steuerung der EndpunktAuslastung wird ein gemeinsamer RateLimiter eingesetzt; transiente Fehler (etwa RateLimits oder Timeouts) werden mit ansteigender Wartezeit wiederholt, deterministische Fehler hingegen unmittelbar protokolliert.

Die Auswertung erfolgt anschließend automatisiert: Aus den protokollierten Rohdaten werden je Auswertungsline die beschriebenen Kennzahlen berechnet und zur weiteren Analyse [PLATZHALTER: verwendete Auswertungs/Visualisierungswerkzeuge, z. B. pandas] aufbereitet.

1.7 Gütekriterien und methodische Einschränkungen

Sämtliche Bewertungen sind Urteile eines LLMJudge und damit modellabhängig; Ergebnisse verschiedener JudgeModelle sind nicht unmittelbar vergleichbar. Die selbst definierten GEvalKriterien sind projektspezifisch und wurden [PLATZHALTER: Angabe, ob und wie eine Validierung, z. B. eine manuelle Nachbewertung einer Stichprobe, durchgeführt

wurde] abgesichert. Simulator und RAGTutor beruhen auf demselben Basismodell, was durch die auf dem verwendeten Endpunkt verfügbaren Modelle bedingt ist; die methodisch zentrale Trennung zwischen Judge und System under Test bleibt hingegen gewahrt. Da ein Teil der Antworten ohne abgerufenen Kontext erzeugt wird, sind die kontextbezogenen Metriken nur eingeschränkt aussagekräftig und werden entsprechend gesondert behandelt. Aufgrund der begrenzten Anzahl an Szenarien je Konfiguration sind die Ergebnisse als explorativ zu verstehen.